6.100L Recitation 6 (21 October 2022)

6.100L 背诵 6（2022 年 10 月 21 日）

Reminders:

提醒：

MQ6 next Monday 10/24

MQ6 下周一 10/24

PS3 halfway hand-in due next Wednesday 10/26

PS3 半途交期将于下周三 10 月 26 日

<u>Lecture 11:</u>

第十一讲：

<u>Aliasing & Cloning</u>

别名 & 克隆

Mutable objects can be changed after they are created.

可变对象在创建后可以更改。

What mutable types do we know so far? Lists.

到目前为止，我们知道哪些可变类型？ 列表。

Aliasing: When two variable *names* refer to the same object.

别名： 当两个变量*名称*引用同一对象时 。

Cloning: Making a copy of an object (typically the safer option).

克隆： 创建对象的副本 （通常是更安全的选项）。

Example 1: Here we are not actually changing the copy of word. Why? Because string are immutable.

示例 1： 这里我们实际上并没有更改 word 的副本。为什么？因为 string 是不可变的。

```
word = “the”
word = “的”

word_copy = word
word_copy = 单词

word += “ bird”
单词 += “鸟”

print(word) # “the bird”
print（word） #“鸟”

print(word_copy) # “the”
print（word_copy） #“的”
```

Example 2:

示例 2：

```
a = [1,2,3,4]
b = a
b += [5]
print(b) # [1,2,3,4,5]
打印 （b） # [1,2,3,4,5]

print(a) # [1,2,3,4,5]
打印 （a） # [1,2,3,4,5]
```

Now b points to a. Since a list is mutable, if you make changes to b, you will change a.

现在 b 指向 a。由于列表是可变的，因此如果对 b 进行更改，则会更改 a。

For immutable types, “=” creates a new object.

对于不可变类型 ，“=” 创建一个新对象 。

For mutable types, “=” will assign the new variable to the same object.

对于可变类型 ，“=” 会将新变量分配给同一个对象。

Why does mutability matter?

为什么可变性很重要?

Makes your code do unexpected things. For example, you may change a variable you did not want to change.

使代码执行意外作 。例如 ， 您可以更改不想更改的变量。

How can I avoid mutability problems?

如何避免可变性问题?

Make clones, or copies.

制作克隆或副本。

```
List_copy = list[:]
List_copy = 列表[:]
```

```
List_copy = list.copy()
List_copy = list.copy ()
```

```
List_copy = copy.copy(list)
List_copy = copy.copy (list)
```

> Shallow vs Deep copy: shallow copy created new data structurem but actual elements are shared – i.e.top level copy only.
>
> 浅层复制与深层复制：浅层复制创建了新的数据结构， 但实际元素是共享的 —— 即仅顶层复制。
>
> ```
> copy.copy(example_list)  # this is a shallow copy
> copy.copy (example_list)   # 这是一个浅拷贝
>
> copy.deepcopy(example_list)  # this is a deepcopy
> copy.deepcopy (example_list)   # 这是一个 deepcopy
> ```

Useful reminder: Don’t change lists while iterating over them!!

有用的提醒： 在迭代列表时不要更改列表！！

`sort` vs `sorted`

排序 VS 排序

`sort`: mutate the list, return nothing

`sort`：更改列表，不返回任何内容

`sorted`: doesn’t mutate the list, return a new sorted list

`sorted`：不改变列表，返回一个新的排序列表

Useful List Methods

有用的列表方法

```
my_list.copy()  # no mutation - returns copy
my_list.copy（）  # 无 mutation - 返回 copy

my_list.reverse()  # mutation
my_list.reverse（）  # 突变

sorted(my_list)  # no mutation – returns sorted list
sorted（my_list）  # no mutation – 返回排序列表

my_list.sort()  # mutation
my_list.sort（）  # 变更

my_list.extend([x,y])  # mutation
my_list.extend（[x, y]）  # 突变

my_list[:]  # makes clone
my_list[：]  # 进行克隆

my_list.remove(2) # mutation
my_list.remove（2）  # 变更

my_list.pop()  # pops last element - mutation
my_list.pop（）  # 弹出最后一个元素 - mutation

my_list.pop(2)  # pops 3rd element
my_list.pop（2）  # 弹出第三个元素

my_list.insert(1, 7)  # inserts 7 in the 2nd position - mutation
my_list.insert（1,  7）  # 在第 2 个位置插入 7 - mutation
```

Lecture 12:

第 12 讲：

List Comprehension

列表推导式

This is a shorter way to create a new list based on the values of an existing data structure.

这是根据现有数据结构的值创建新列表的较短方法。

```
# standard method
# 标准方法

    fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
    水果 = [“苹果”, “香蕉”, “樱桃”, “猕猴桃”, “芒果”]

    new_list = []
    for x in fruits:  # standard for loop
    for 水果中的 x :   # 标准 for 循环

      if "a" in x:
       如果 x:

        newlist.append(x)
        newlist.append (x)  的

  # using list comprehension
# 使用列表推导式

    newlist = [x for x in fruits if "a" in x]
    newlist = [x for x in fruits if“a”in x]
```

<u>Default Parameters in Functions</u>

函数中的默认参数

Example: here y is a default parameter

示例：这里 y 是默认参数

```
def multiply(x, y=2):
def multiply (x,  y=2):

 output = x * y
输出 = x * y

 return output
返回输出
```

```
print(multiply(3)) # outputs 6
print（multiply（3）） # 输出 6

print(multiply(3,4)) # outputs 12
print（multiply（3,4）） # 输出 12
```

<u>Testing</u>

测试

Write code that can be broken up into parts and tested easily (inc. comments + assumptions)

编写可拆分为多个部分并易于测试的代码（包括注释 + 假设）

3 Class of tests:

3 测试类别：

Unit testing - test each function separately

单元测试 - 单独测试每个功能

Regression testing - add tests for bugs as you find them

回归测试 - 在发现错误时添加测试

Integration testing - high level - does the program do what you want it to

集成测试 - 高级 - 程序是否按您的要求工作

Main 2 testing approaches:

主要 2 种测试方法：

Black box testing - designed without looking at code, avoids implementer bias, can be reused if implementation changes.

黑盒测试 - 无需查看代码即可设计，避免了实现者的偏见，如果实现发生变化，可以重用。

Glass box testing - use code to guide design of test cases.

玻璃盒测试 - 使用代码来指导测试用例的设计。

Remember to test edge cases!

记得测试边缘情况！

<u>Debugging</u>

调试

General tips:

一般提示：

Print out the values of your variables

打印出变量的值

Google is your friend if you encounter an error you don't understand.

如果您遇到不理解的错误，Google 是您的朋友。

The stack trace shows what line(s) caused the error -- use it!

堆栈跟踪显示导致错误的行 -- 使用它！

Common error messages

常见错误消息

```
test = [1,2,3]
```

```
测试 = [1,2,3]
```

```
test[4] # will throw an IndexError since there doesn't exist an element at index 4
```

```
test[4] # 将抛出一个 IndexError，因为在索引 4 处不存在一个元素
```

```
int(test) # will throw a TypeError since lists cannot be converted into integer
```

```
int（test） # 会抛出一个 TypeError，因为列表不能被转换成整数
```

```
# Any error in Python syntax will throw a SyntaxError
```

```
# Python 语法中的任何错误都会引发 SyntaxError
```

6.100L Introduction to CS and Programming Using Python

6.100L CS 和使用 Python 编程简介

Fall 2022

2022 年秋季

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

有关引用这些材料或我们的使用条款的信息，请访问：https://ocw.mit.edu/terms